Расширение меньшего класса

Д.В. Парфенов

Пусть в задаче классификации первый класс значительно меньше второго: $|C_1| \ll |C_2|$, что приводит к дефектам обучения. Наша цель - нарастить его до соизмеримых с $|C_2|$ (например, равных) размеров нетривиальным образом, поскольку дублированные объекты не повышают качества решателя, не внося разнообразия в обучающие данные, а лишь номинально увеличивают размер класса. Расширение класса должно следовать общей его топологии в пространстве признаков размерности M в том смысле, что новые объекты, которые назовём *синтетическими*, должны быть расположены внутри непересекающихся кластеров C_1, в которых нет ни одного объекта из C_2. Такое поведение обеспечивается размещением синтетических объектов внутри выпуклых оболочек кластеров своего класса, которые, однако, очень трудно практически найти в пространстве признаков высокой размерности. Поэтому будем использовать не геометрический, а алгебраический подход. Решение разбивается на два шага:

- Разбиение C_1 на кластеры, которые формируются жадным наращиванием по принципу топологической близости;
- Генерация синтетических объектов как взвешенных ограниченных линейных комбинаций объектов кластеров.

Каждый элемент размеченной обучающей выборки имеет вид $\mathbf{c}_{p,q} = (c_{p,q,1}, c_{p,q,2}, \ldots, c_{p,q,M})^T$, где p – номер класса, q – номер объекта в этом классе. Такой нестандартный способ индексации используется далее систематически для удобства перехода между поэлементной и матричной записью, отличающимися только наличием или отсутствием последнего индекса. Каждый класс или его кластер можно рассматривать как совокупность имеющих равную высоту вектор-столбцов признаков, т.е. как прямоугольную матрицу. Пусть Q_k – матрица объектов k-ого кластера, $k = 1, \ldots, K$, состоящего из N_k объектов класса C_1. Объект $\mathbf{c}_{p,q}$ расположен внутри выпуклой оболочки этого кластера, если для него справедливо:

$$Q_k \mathbf{a} = \mathbf{c}_{p,q}, \tag{1}$$

$$\sum_{n=1}^{N_k} a_n = 1, \quad a_n \geq 0, \quad n = 1, \ldots, N_k. \tag{2}$$

Свойство (1) назовём выразимостью $\mathbf{c}_{p,q}$ через Q_k, а (2) - вложением $\mathbf{c}_{p,q}$ в выпуклую оболочку кластера Q_k.

Предлагается следующий алгоритм:

1) Счётчик кластеров объектов первого класса $K = 1$.

2) Создаём новый кластер Q_K. Исходно положим пустым $\bar{C}_1 = \emptyset$ множество непригодных для включения в этот кластер элементов C_1. Среди объектов C_1 сообразно выбранному расстоянию найдётся ближайший к классу C_2 в пространстве признаков (если таких объектов несколько, можно взять любой из них). Думается, лучшим выбором метрики является расстояние между ближайшими элементами в смысле кластеризации методом SLINK: $||\mathbf{c}_{1,*} - \mathbf{c}_{2,*}|| = \min_{i=1\ldots|C_1|,\ j=1\ldots|C_2|} ||\mathbf{c}_{1,i} - \mathbf{c}_{2,j}||$, $\mathbf{c}_{1,*} \in C_1$ и $\mathbf{c}_{2,*} \in C_2$. Вектор признаков $\mathbf{c}_{1,*}$ становится первым столбцом в матрице Q_K признаков K-ого кластера объектов C_1: $Q_{K,1} = \mathbf{c_{1,*}}$, а из самого класса этот объект удаляется как уже использованный: $C_1 = C_1 \setminus \mathbf{c}_{1,*}$. Текущее количество столбцов в Q_K обозначим N_K, пока $N_K = 1$.

3) Берём очередной ближайший к кластеру Q_K объект $\mathbf{c}_{1,*}$ из C_1, удаляем его из необработанной части класса: $C_1 = C_1 \setminus \mathbf{c}_{1,*}$ и "условно" помещаем как новый правый столбец в матрицу Q_K: $Q_K = Q_K \cup \mathbf{c}_{1,*}$. Таким образом кластер расширился до $N_K = N_K + 1$ элементов.

4) Выполняем процедуру проверки "годности" кластера Q_K с новым элементом. Для каждого $c_{2,*} \in C_2$:

   a) Решаем (переопределённую при $N_K < M$ и линейной независимости признаков элементов в Q_K) СЛАУ вида

$$\begin{pmatrix} Q_{K,1,1} & Q_{K,2,1} & \ldots & Q_{K,N_K,1} \\ Q_{K,1,2} & Q_{K,2,2} & \ldots & Q_{K,N_K,2} \\ \vdots & \vdots & \ddots & \vdots \\ Q_{K,1,M} & Q_{K,2,M} & \ldots & Q_{K,N_K,M} \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_{N_K} \end{pmatrix} = \begin{pmatrix} c_{2,*,1} \\ c_{2,*,2} \\ \vdots \\ c_{2,*,M} \end{pmatrix},$$

   сокращённо:

$$Q_K \mathbf{a} = \mathbf{c}_{2,*} \tag{3}$$

относительно $\mathbf{a}$. Но вследствие (2) один коэффициент (пусть это будет a_{N_K}) и соответствующий ему последний столбец Q_K – лишние. Подставляя (2) в (3):

$$\begin{pmatrix} Q_{K,1,1}-Q_{K,N_K,1} & \cdots & Q_{K,N_K-1,1}-Q_{K,N_K,1} \\ Q_{K,1,2}-Q_{K,N_K,2} & \cdots & Q_{K,N_K-1,2}-Q_{K,N_K,2} \\ \vdots & \ddots & \vdots \\ Q_{K,1,M}-Q_{K,N_K,M} & \cdots & c_{K,N_K-1,M}-Q_{K,N_K,M} \end{pmatrix} \begin{pmatrix} \hat{a}_1 \\ \hat{a}_2 \\ \vdots \\ \hat{a}_{N_K-1} \end{pmatrix} =$$

$$\begin{pmatrix} c_{2,*,1}-Q_{K,N_K,1} \\ c_{2,*,2}-Q_{K,N_K,2} \\ \vdots \\ c_{2,*,M}-Q_{K,N_K,M} \end{pmatrix},$$

сокращённо:

$$\hat{Q}_K\hat{\mathbf{a}} = \hat{\mathbf{c}}_{2,*}. \qquad (4)$$

Возникает вопрос, не логично ли при построении $\hat{Q}_K$ из Q_K вычитать не последний, а первый столбец:

$$\begin{pmatrix} Q_{K,2,1}-Q_{K,1,1} & \cdots & Q_{K,N_K,1}-Q_{K,1,1} \\ Q_{K,2,2}-Q_{K,1,2} & \cdots & Q_{K,N_K,2}-Q_{K,1,2} \\ \vdots & \ddots & \vdots \\ Q_{K,2,M}-Q_{K,1,M} & \cdots & c_{K,N_K,M}-Q_{K,1,M} \end{pmatrix} \begin{pmatrix} \hat{a}_2 \\ \hat{a}_3 \\ \vdots \\ \hat{a}_{N_K} \end{pmatrix} =$$

$$\begin{pmatrix} c_{2,*,1}-Q_{K,1,1} \\ c_{2,*,2}-Q_{K,1,2} \\ \vdots \\ c_{2,*,M}-Q_{K,1,M} \end{pmatrix}.$$

С точки зрения объёма вычислений это оправдано, поскольку при добавлении нового стобца не нужно пересчитывать остальную часть столбцов матрицы $\hat{Q}_K$. Однако в (4) каждый следующий столбец отличается от предыдущих по построению всё значительнее (см.пункт 3) алгоритма). Это позволяет избежать вычитания близких чисел, как известно, являющегося классическим источником лавинно нарастающей погрешности при использовании вычислений с плавающей точкой. Этот вопрос стоит изучить, если метод в целом окажется рабочим.

б) Для всех элементов решения $\hat{\mathbf{a}}$: если $\hat{a}_m < 0$, установить $\hat{a}_m = 0$. Это достигается исключением соответствующих столбцов $\hat{Q}_K$ и решением редуцированного (4) до тех пор, пока либо не

исчезнут все отрицательные компоненты $\mathbf{a}$, либо в Q_K не останется два столбца. В последнем случае при $|\hat{a}_1 + \hat{a}_2 - 1| > \zeta$ или $\min\{\hat{a}_1, \hat{a}_2\} < -\zeta$ устанавливаем флаг заведомой непринадлежности $\hat{\mathbf{c}}_{2,*}$ кластеру Q_K: $cluster_is_good = true$. Иначе $cluster_is_good = unknown$. Удаление стобцов в $\hat{Q}_K$ никак не влияет на дальнейшие вычисления, поскольку предполагается, что матрица $\hat{Q}_K$ – врѐменная, каждый раз заново расчитываемая на основе Q_K.

в) Если выполнены оба условия нахождения $\mathbf{c}_{2,*}$ внутри выпуклой оболочки элементов Q_K

$$||\hat{Q}_K\hat{\mathbf{a}} - \hat{\mathbf{c}}_{2,*}|| < \xi \quad \text{and} \quad \text{not}(cluster_is_good == true), \quad (5)$$

необходимо изъять из кластера Q_K только что "условно" добавленный элемент $\mathbf{c}_{1,*}$: $Q_K = Q_K \setminus \mathbf{c}_{1,*}$, $N_K = N_K - 1$ и переместить его во множество негодных для этого кластера $\bar{C}_1 = \bar{C}_1 \cup \mathbf{c}_{1,*}$.

Если хотя бы одно из условий (5) нарушено, утвердить "условно" добавленный в пункте 3) объект $\mathbf{c}_{1,*}$ в кластере: Q_K.

СЛАУ (4) устроена таким образом, что при добавлении нового вектора признаков в Q_K длина вектора $\mathbf{a}$ увеличивается на единицу, а равное M число уравнений остаётся неизменным. При достаточной мощности класса $|C_1|$ это рано или поздно приведёт к недоопределённости СЛАУ, что повлечёт ложноположительные результаты проверки (5) на способность находящейся внутри выпуклой оболочки комбинации объектов Q_K породить объект из C_2. Впрочем, эти ошибочные отклонения вставки в кластер нового объекта $\mathbf{c}_{1,*}$ не станут причиной неправильной работы всего алгоритма, а только приведут к бесполезным вычислениям. Если признаки разных объектов являются линейно независимыми, (5) станет неодоопределённой при $M+1$ столбцах. Нужно проверить гипотезу о том, что если неравенство в (5) выполняется при добавлении некоторого $\mathbf{c}_{1,*}$ для всех $\mathbf{c}_{2,*}$, это означает переопределённость (4). Это дало бы естественный, простой и адаптивный способ досрочно и вычислительно экономно завершить формирование очередного кластера. Ещё более простое, но грубое решение состоит в сравнении $M \leq N_K$ в предположении линейной независимости признаков элементов кластера Q_K.

Множественное численное решение (4) довольно трудозатратно, поэтому стоит единожды для каждого добавляемого $\mathbf{c}_{1,*}$ вычислять псевдообратную к $\hat{Q}_K$ матрицу и далее для каждо-

го проверяемого $\mathbf{c}_{2,*}$ алгебраически эквивалентно (4) вычислять коэффициенты с помощью умножения матрицы на вектор: $\hat{\mathbf{a}} = \mathrm{pinv}(\hat{Q}_K)\hat{\mathbf{c}}_{2,*}$.

5) Если $C_1 \neq \emptyset$ и СЛАУ (4) не является недоопределённой, пытаемся добавить очередной элемент в Q_K, для чего переходим к шагу 3) алгоритма.

   Если выполнено только $C_1 \neq \emptyset$, начинаем формировать новый кластер из объектов, не попавших в предыдущие кластеры: $C_1 = C_1 \cup \bar{C}_1$, $\bar{C}_1 = \emptyset$, $K = K + 1$ и переходим к шагу 2).

6) Инвариант: $C_1 = \emptyset$, весь меньший класс кластеризован на "безопасные" кластеры в смысле невозможности порождения ими элементов из C_2. При этом по построению чем меньше номер кластера k, $k = 1, \ldots, K$, тем более важен кластер при обучении, поскольку его элементы характеризуются меньшими расстояниями до C_2. Поэтому из необходимых $|C_2| - |C_1|$ синтетических объектов, вероятно, бо́льшую часть стоит получить из кластеров с меньшими k. Способ оптимального распределения новых генерирумых представителей C_1 между кластерами требует дополнительного исследования. Следующий адаптивный алгоритм исходно представляется неплохим:

   а) Для всякого кластера Q_k, $k = 1, \ldots, K$ рассчитать расстояния до C_2 (хорошим выбором снова видится SLINK). Число произведённых этим кластером синтетических объектов S_k логично сделать обратно пропорциональным расстоянию $Q_k - C_2$:

$$S_k = \frac{|C_2| - |C_1|}{||Q_k - C_2|| \sum_{l=1}^{K} [1/||Q_l - C_2||]}.$$

   б) Для Q_k каждый синтетический объект получается как линейная комбинация входящих в кластер: $\mathbf{c}_{1,*} = \sum_{n=1}^{N_k} a_n Q_{k,n}$ с ограниченными случайными весами $\sum_{n=1}^{N_k} a_n = 1$, $a_n \geq 0$, $n = 1, \ldots, N_k$. Хорошим выбором весов a_n является исходная генерация равномерно распределённых на $[0, 1)$ случайных чисел с последующей их нормировкой на $\sum_{n=1}^{N_k} a_n$.

Если алгоритм окажется работоспособным, возможно его обобщение на большее число классов. Гиперпараметры:

1) Функция расстояния между объектами в пространстве признаков $||\mathbf{c}_{p,q} - \mathbf{c}_{r,s}||$. Логично пользоваться центрированным и нормированным пространством признаков, в котором естественен выбор l_p расстояния $\sqrt[p]{\sum_{m=1}^{M} |\mathbf{c}_{p,q,m} - \mathbf{c}_{r,s,m}|^p}$. Особенно интересны случаи $l = 1, 2, \infty$.

2) Функция расстояния между объектом и классом (кластером) в пространстве признаков $||\mathbf{c}_{p,q} - C_r||$. Здесь разумным выбором представляются расстояние метода кластеризации SLINK или расстояние Хаусдорфа (Hausdorff distance).

3) Функция расстояния между классом и другим классом (кластером) в пространстве признаков. Здесь также хорошими вариантами видятся расстояние метода кластеризации SLINK или расстояние Хаусдорфа (Hausdorff distance).

4) Гиперпараметры уменьшения чувствительности к погрешностям плавающей точки $\zeta > 0$ и $\xi > 0$ берутся очень малыми, но ненулевыми. При чрезмерно больших ξ или ζ могут быть ошибочно исключены новые элементы кластера Q_k. Наоборот, слишком малые их значения вследствие погрешностей вычислений с плавающей точкой не позволят правильно отбраковать $\mathbf{c}_{1,*}$ или обнаружить недоопределённость системы (4). Предположительно, при использовании плавающей точки с двойной точностью практический диапазон значений этих гиперпараметров составляет $10^{-5}...10^{-9}$.